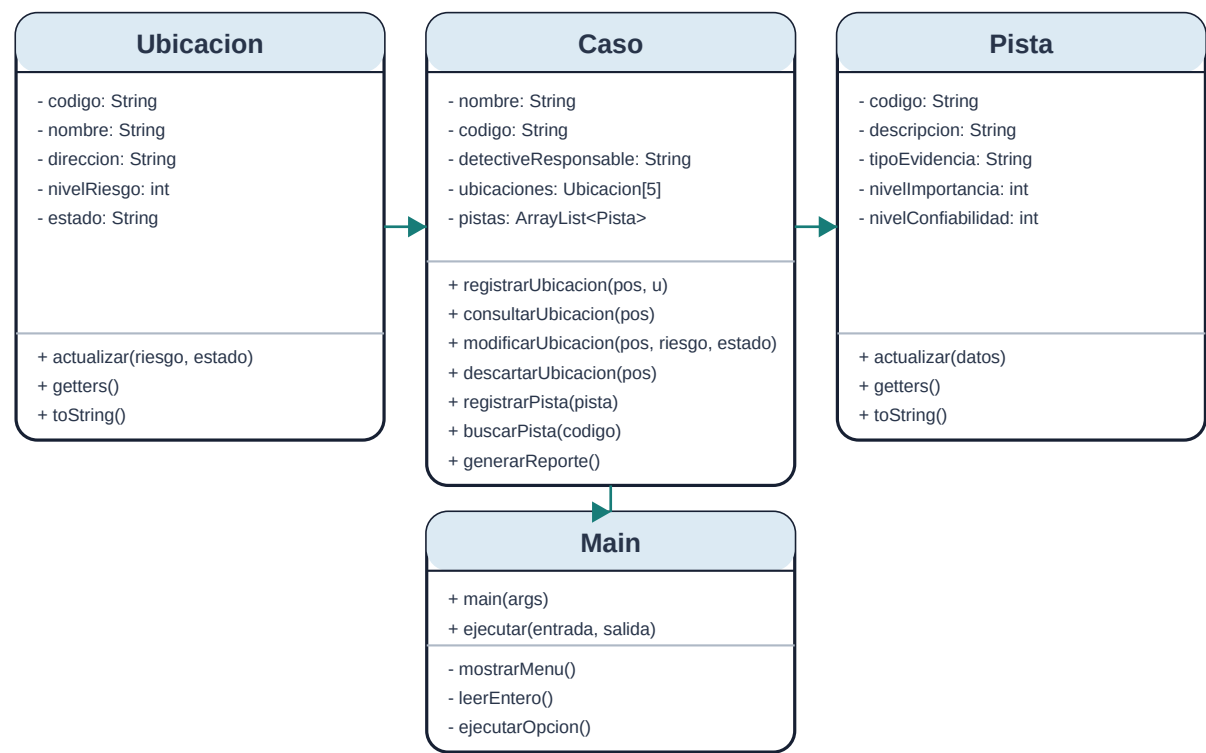


LABORATORIO 2

Arreglos y excepciones



AGENCIA DE DETECTIVES - CASO MISTERIOSO

Documento de análisis, diseño y decisiones de implementación

Curso	Programación Orientada a Objetos
Tema	Arreglos de objetos, ArrayList y excepciones
Programa	Java de consola
Estructuras principales	Ubicacion[5] y ArrayList

1. Objetivo y alcance

El programa organiza la información de una agencia privada de detectives, el caso trabaja con un arreglo de cinco espacios para ubicaciones y con un ArrayList para las pistas, también se validan los datos y se controlan los errores de entrada para que el menú continúe

Criterios de alcance

- El programa funciona en consola y no depende de bibliotecas externas
- Las posiciones vacías del arreglo se conservan como null y se revisan antes de usar sus métodos
- Las pistas se consultan y modifican recorriendo el ArrayList por código
- El menú continúa después de errores que se pueden recuperar

2. Requisitos funcionales

ID	Requisito
RF-01	Solicitar nombre del caso, código de identificación y detective responsable
RF-02	Administrar un máximo de cinco ubicaciones en un arreglo de objetos
RF-03	Guardar código, nombre, dirección, nivel de riesgo y estado de cada ubicación
RF-04	Validar el nivel de riesgo entre 1 y 10 al crear o modificar una ubicación
RF-05	Registrar una ubicación en la posición indicada, revisando límites, disponibilidad y datos
RF-06	Mantener null en las posiciones del arreglo que todavía están vacías
RF-07	Consultar las ubicaciones sin mostrar las posiciones null como existentes
RF-08	Consultar una ubicación específica e informar si la posición es inválida o está vacía
RF-09	Modificar el nivel de riesgo y el estado de una ubicación existente
RF-10	Descartar una ubicación y devolver su posición a null
RF-11	Guardar una cantidad desconocida de pistas directamente en ArrayList
RF-12	Guardar código, descripción, tipo de evidencia, importancia y confiabilidad de cada pista
RF-13	Validar importancia entre 1 y 10 y confiabilidad entre 0 y 100
RF-14	Registrar pistas y rechazar códigos repetidos
RF-15	Consultar todas las pistas y manejar una colección vacía
RF-16	Buscar una pista por su código recorriendo el ArrayList
RF-17	Modificar una pista existente aplicando sus validaciones
RF-18	Eliminar una pista localizada por código
RF-19	Calcular cantidades, máximos y promedio para ubicaciones y pistas
RF-20	Manejar InputMismatchException al solicitar números con Scanner
RF-21	Usar try-catch y finally para mantener activo el menú después de errores
RF-22	Incluir un driver program con un menú de 13 opciones

3. Análisis de clases

Clase	Responsabilidad
Caso	Guarda los datos del caso y maneja las ubicaciones y las pistas
Ubicacion	Guarda los datos de un lugar y valida el nivel de riesgo
Pista	Guarda los datos de una evidencia y valida importancia y confiabilidad
Main	Muestra el menú, lee los datos y controla los errores de entrada

3.1 Atributos

Clase	Atributo	Tipo	Uso
Caso	nombre, codigo, detectiveResponsable	String	Identificar la investigación y a su responsable
Caso	ubicaciones	Ubicacion[]	Arreglo fijo de cinco posiciones, los espacios vacíos contienen null
Caso	pistas	ArrayList	Colección dinámica para las evidencias encontradas
Ubicacion	codigo, nombre, direccion, estado	String	Describir e identificar el lugar
Ubicacion	nivelRiesgo	int	Representar el riesgo validado de 1 a 10
Pista	codigo, descripcion, tipoEvidencia	String	Identificar y describir la evidencia
Pista	nivelImportancia	int	Valorar la evidencia de 1 a 10
Pista	nivelConfiabilidad	int	Valorar la confiabilidad de 0 a 100

3.2 Encapsulamiento

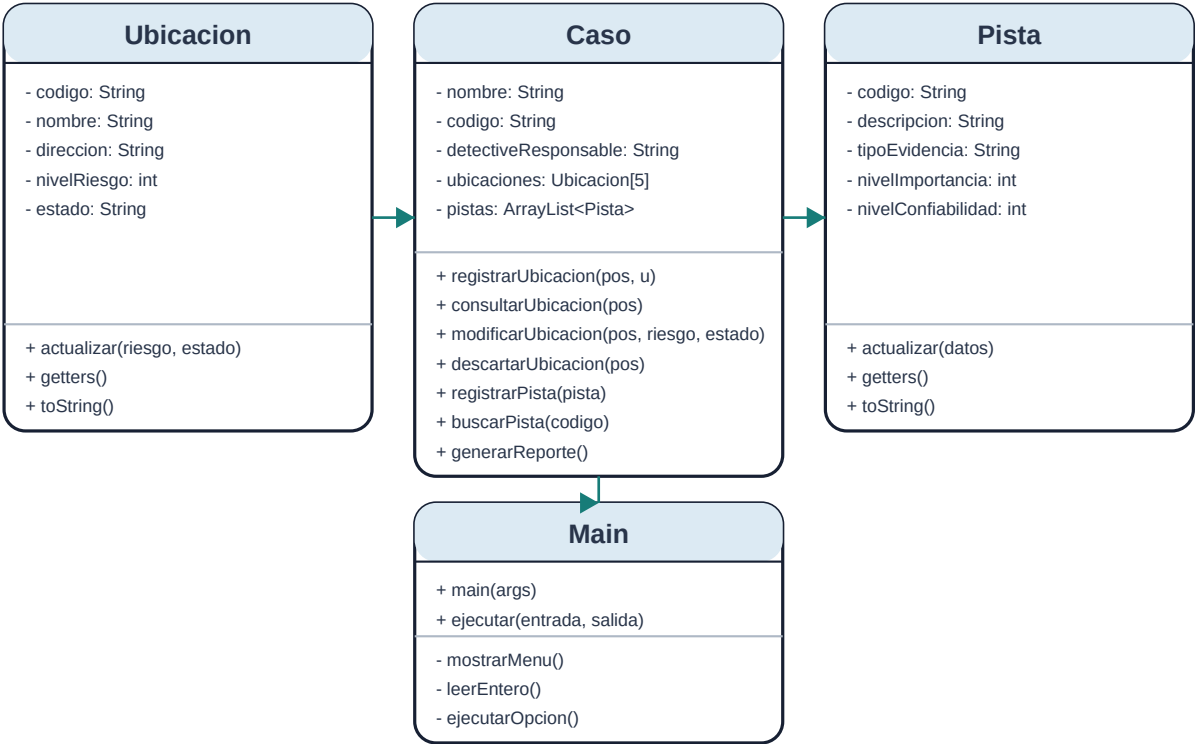
Los atributos son privados, las clases de dominio tienen getters para consultar los datos y métodos de operación para modificar solo los valores permitidos, las validaciones se hacen antes de cambiar el estado

4. Métodos y excepciones

Clase	Método	Resultado y validaciones
Caso	registrarUbicacion(posicion, ubicacion)	Revisa límites, null y disponibilidad antes de asignar el objeto
Caso	consultarUbicaciones() / consultarUbicacion(posicion)	Recorre el arreglo y no usa métodos sobre posiciones null
Caso	modificarUbicacion / descartarUbicacion	Rechaza posiciones inválidas o vacías
Caso	registrarPista / buscarPista	Recorre ArrayList y evita códigos duplicados
Caso	modificarPista / eliminarPista	Busca por código y rechaza pistas inexistentes
Caso	generarReporte()	Calcula cantidades, máximos y promedio sin operar sobre una colección vacía
Ubicacion	actualizar(riesgo, estado)	Valida riesgo 1-10 y estado no vacío
Pista	actualizar(datos)	Valida importancia 1-10 y confiabilidad 0-100
Main	leerEntero()	Captura InputMismatchException, limpia el token y vuelve a solicitar
Main	ejecutarOpcion()	Captura IllegalArgumentException y deja disponible el menú

El bloque finally se ejecuta al terminar cada operación del menú, incluso cuando una validación produce una excepción, así el usuario recibe el mensaje de cierre de la operación y puede continuar

5. Diseño UML



6. Menú del programa

Opción	Operación
1	Nuevo caso, reemplaza el caso actual y reinicia sus estructuras
2-6	Registrar, consultar, modificar y descartar ubicaciones
7-11	Registrar, consultar, buscar, modificar y eliminar pistas
12	Mostrar el reporte de investigación con los cálculos pedidos
13	Salir de la aplicación

7. Verificación propuesta

- Compilar todas las clases con el JDK sin dependencias externas
- Crear un caso y registrar una ubicación en una posición disponible
- Intentar usar una posición inválida y confirmar que el programa continúa
- Ingresar texto donde se solicita un número y confirmar la recuperación del Scanner
- Registrar dos pistas, comprobar búsqueda, modificación, eliminación y reporte
- Confirmar que el reporte vacío no intenta calcular máximos ni promedio

Conclusión

La solución separa los datos de la interacción, conserva el arreglo y el ArrayList solicitados y coloca las reglas de validación en las clases que protegen cada objeto, el menú queda sencillo de seguir y las operaciones se pueden comprobar por separado